1. Prozesuak Javan

ProcessBuilder-ek prozesu berri bat prestatzen du (programa + argumentuak); `start()`-ek abiarazten du eta **Process** objektu bat itzultzen du.

```
ProcessBuilder pb = new ProcessBuilder("cmd", "/c", "echo", "Kaixo");
pb.directory(new File("C:\\temp"));   // lan-karpeta
Process p = pb.start();                 // prozesu umea abiarazi
```

Birbideratzeak (lehenetsia: kontsola):

Metodoa	Zer birbideratzen du
`redirectInput(File)`	Umearen sarrera (fitxategitik irakurri)
`redirectOutput(File)`	Umearen irteera (fitxategira idatzi)
`redirectError(File)`	Umearen erroreak

```
ProcessBuilder pb = new ProcessBuilder("java", "-cp", "bin", "Oihartzuna");

// Irakurri umeari emateko

pb.redirectInput(new File("sarrera.txt"));

// Umearen emaitza fitxategain idazten du

pb.redirectOutput(new File("irteera.txt"));

 // Umearen erroreak idazten ditu

 pb.redirectError(new File("erroreak.txt"));

 Process p = pb.start();

 int kodea = p.waitFor()  ;

 System.out.println("Irteera kodea: " + kodea);
```

Process-en streamak (kontuz: izena gurasoaren ikuspuntutik dago):

Metodoa	Zertarako
`getInputStream()`	Umeak idazten duena irakurri (bere irteera)
`getErrorStream()`	Umearen erroreak irakurri
`getOutputStream()`	Umearen sarrerara idatzi

```
Process p = new ProcessBuilder("java", "Oihartzuna").start();

// Umeari datuak bidali
OutputStream os = p.getOutputStream();
os.write("Ane\n".getBytes());
os.close();

// Umearen erantzuna irakurri
BufferedReader br = new BufferedReader(
        new InputStreamReader(p.getInputStream()));
String lerroa;
while ((lerroa = br.readLine()) != null)
    System.out.println(lerroa);

// Erroreak irakurri
BufferedReader be = new BufferedReader(
        new InputStreamReader(p.getErrorStream()));
while ((lerroa = be.readLine()) != null)
    System.out.println("ERROREA: " + lerroa);
```

Process-en beste metodoak:

Metodoa	Zer itzultzen / egiten du
`waitFor()`	Amaitu arte itxaron; irteera-kodea (int)
`waitFor(n, TimeUnit)`	Gehienez n itxaron; true amaitu bada
`exitValue()`	Irteera-kodea
`isAlive()`	true oraindik exekutatzen ari bada
`destroy()` / `destroyForcibly()`	Prozesua hil

Runtime: JVMaren ingurunea.

```
Runtime rt = Runtime.getRuntime();
int cpu = rt.availableProcessors();             // JVMaren nukleoak
long erabilia = rt.totalMemory() - rt.freeMemory(); // okupatutako memoria
rt.gc();                                         // Garbage Collector-a deitu
```

```
Process p2 = rt.exec("notepad");                  // prozesu bat abiarazteko beste modu
bat
```

Bi prozesuren arteko komunikazioa (6. ariketa) ;

```
package Procesos;


import java.io.BufferedReader;

import java.io.File;

import java.io.InputStreamReader;

import java.io.OutputStream;


public class ProcesoA {


	public static void main(String[] args) {

		// TODO Auto-generated method stub

			try {

				ProcessBuilder pb = new ProcessBuilder("java",
"arik_6.ProcesoB");

				pb.directory(new File("bin")); // Directorio con las clases
compiladas

				Process pB = pb.start();


				OutputStream os = pB.getOutputStream();//Escribir a procesoB

				BufferedReader br = new BufferedReader(new
InputStreamReader(pB.getInputStream()));//Leer procesoB


				os.write("Nola deitzen zara?\n".getBytes());

				os.flush();

				System.out.println("Respuesta B: " + br.readLine());

				// Enviar pregunta 2

				os.write("Zenbat urte dauzkazu?\n".getBytes());

				os.flush();
```

```
                        System.out.println("Respuesta B: " + br.readLine());



                } catch (Exception e) {
                        System.out.println("Error al lanzar el proceso");
                }
        }

}

package Procesos;
import java.util.Scanner;
public class ProcesoB {
        public static void main(String[] args) {
                Scanner sc = new Scanner(System.in);
                while (sc.hasNextLine()) {
                        String preg = sc.nextLine();
                        if (preg.equals("Nola deitzen zara?")) {
                                System.out.println("Nire izena ProcesoB da");
                        } else if (preg.equals("Zenbat urte dauzkazu?")) {
                                System.out.println("20 urte dauzkat");
                        } else {
                                System.out.println("Ez dut galdera hori ulertzen");
                        }
                        System.out.flush(); // Asegurar respuesta inmediata al padre
                }
        }
}
```

2. Hariak (Threads)

Haria prozesu baten barruan beste zeregin batzuekin batera exekutatzen den zeregin bat da. Prozesu bateko hariek kodea eta datuak partekatzen dituzte; bakoitzak bere pila du.

Haria sortzeko bi modu:

```
// 1) Thread-etik heredatuz
class Agurra extends Thread {
    public void run() {
        System.out.println("Kaixo Hugo Vicente Vallejo " + getName() + "-tik");
    }
}
Agurra a = new Agurra();
a.start();

// 2) Runnable inplementatuz (beste klase batetik heredatzea uzten du)
class Lana implements Runnable {
    public void run() {
        System.out.println("Lanean...");
    }
}
Thread t = new Thread(new Lana());
t.start();
```

Thread-en metodo erabilienak:

Metodoa	Zer egiten du
`start()`	Haria abiarazi (`run()` hari berri batean)
`run()`	Hariak exekutatzen duen kodea
`sleep(ms)`	Uneko haria ms milisegundoz lo (static)
`join()`	Deitzen duen haria hau amaitu arte zain
`isAlive()`	true haria exekutatzen ari bada
`interrupt()`	Hariari etenaldi bat bidali
`setName()` / `getName()`	Izena ezarri / irakurri
`setPriority()` / `getPriority()`	Lehentasuna ezarri / irakurri (1-10)

3. Harien kudeaketa

Haria amaitu: `stop()`, `destroy()`, `suspend()` eta `resume()` **zaharkituta** daude. Zuzena: `run()`-eko begizta boolean batekin kontrolatzea.

```
class Kontagailua extends Thread {
    private volatile boolean jarraitu = true;

    public void gelditu() { jarraitu = false; }

    public void run() {
        int n = 0;
        while (jarraitu) {
            n++;
        }
        System.out.println("Amaiera: " + n);
    }
}

// main:
Kontagailua k = new Kontagailua();
k.start();
Thread.sleep(500);
k.gelditu();
```

Haria eten (interrupt): `interrupt()`-ek hariari abisatzen dio; `sleep()` edo `wait()`-en badago, `InterruptedException` jaurtitzen da. Programatzaileak erabakitzen du zer egin (normalean amaitu).

```
public void run() {
    try {
        while (true) {
            System.out.println("Tik");
            Thread.sleep(1000);
        }
    } catch (InterruptedException e) {
        System.out.println("Etenda, amaitzen");
    }
}
// main-en: h.interrupt();
```

Beste hari baten zain egon join()-ekin:

```
Thread a = new Thread(new Lana());
Thread b = new Thread(new Lana());
a.start();
b.start();
a.join();   // main zain a amaitu arte
b.join();   // main zain b amaitu arte
System.out.println("Denak amaitu dira");
```

`join()`-ek `InterruptedException` jaurti dezake: `main`-ek `throws InterruptedException` behar du edo try/catch.

Lehentasunak: lehenetsita, hari batek gurasoaren lehentasuna heredatzen du.

```
h.setPriority(Thread.MAX_PRIORITY);
System.out.println(h.getPriority());
```

4. Sinkronizazioa

Hainbat harik objektu bat partekatzen dutenean, emaitza exekuzio-ordenaren araberakoa izan daiteke. Hori saihesteko kodea sinkronizatu behar da. Sinkronizazioak errendimendua murrizten du: beharrezkoa denean bakarrik erabili.

Objektu partekatua: hari guztiek eraikitzailean jasotzen dute objektu bera.

```
class Kutxa {
    private int dirua = 50;

    public synchronized void atera(String nor, int kopurua) {
        if (dirua >= kopurua) {
            dirua -= kopurua;
            System.out.println(nor + "-k " + kopurua + " atera ditu. Geratzen: " +
dirua);
        } else {
            System.out.println(nor + ": ez dago nahikoa diru");
        }
    }
}

class Pertsona extends Thread {
    private Kutxa kutxa;
    private int kopurua;

    public Pertsona(String izena, Kutxa kutxa, int kopurua) {
        super(izena);
        this.kutxa = kutxa;
        this.kopurua = kopurua;
    }

    public void run() {
        kutxa.atera(getName(), kopurua);
    }
}

// main:
Kutxa k = new Kutxa();
new Pertsona("Ane", k, 40).start();
new Pertsona("Iker", k, 30).start();
```

synchronized metodoa: objektu berean hari bakarrak exekuta dezake aldi berean; besteak blokeoa askatu arte zain geratzen dira.

synchronized blokea: objektu jakin baten gainean instrukzio batzuk bakarrik sinkronizatzen ditu. Saihestu, metodo osoa sinkroniza badaiteke.

```
synchronized (kutxa) {
    kutxa.atera("Ane", 20);
}
```

volatile: aldagaia beti memoria nagusian gordetzen da, ez prozesadorearen cachean; horrela hari guztiek balio eguneratua ikusten dute berehala.

```
private volatile boolean aktibo = true;
```

wait() / notify(): haria pausatu eta berriz martxan jartzeko. Beti synchronized kodearen barruan.

Metodoa	Zer egiten du
`wait()`	Haria gelditu eta blokeoa askatzen du abisu bat jaso arte
`notify()`	`wait()`-en dagoen hari bat esnatzen du
`notifyAll()`	`wait()`-en dauden hari guztiak esnatzen ditu

```
class Semaforoa {
    private boolean berdea = false;

    public synchronized void berdearenZain() throws InterruptedException {
        while (!berdea)
            wait();          // norbaitek abisatu arte geldirik
        System.out.println(getClass().getSimpleName() + ": aurrera");
    }

    public synchronized void berdeaJarri() {
        berdea = true;
        notifyAll();         // zain daudenak esnatu
    }
}
```